



Федеральное агентство по рыболовству
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Астраханский государственный технический университет»

Система менеджмента качества в области образования, воспитания, науки и инноваций
ООО «ДКС РУС» по международному стандарту ISO 9001:2015

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОММУНИКАЦИЙ
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЗАЩИТА ИНФОРМАЦИИ

ОТЧЕТ

о выполнении лабораторной работы № 4
Реализация электронной цифровой подписи

Выполнил:
студент гр. ЗИНРб-51

_____ Генералов В. В.

«_____» _____ 2026 г.

Преподаватель:

_____ Савоськин И. А.

«_____» _____ 2026 г.

Астрахань – 2026

ВВЕДЕНИЕ

Электронная цифровая подпись (ЭЦП) удостоверяет авторство и неизменность электронного документа. В работе ЭЦП строится из программ предыдущих лабораторных работ: алгоритма шифрования RSA (работа №2) и собственной хэш-функции (работа №3).

Цель работы: «ознакомиться с принципами формирования и проверки электронной цифровой подписи. Получить практические навыки программирования ЭЦП с использованием алгоритма RSA и хэш-функции».

Задание работы состоит из четырёх пунктов:

1. «С помощью программ, выполненных в предыдущих лабораторных, реализовать и запрограммировать алгоритм электронной цифровой подписи с помощью алгоритмов RSA и самостоятельно придуманной хэш-функции.
2. Сообщением, подписанным ЭЦП должен быть текст, поддерживающий все символы ASCII.
3. Программа должна сама подписывать сообщение, а также проверять подпись.
4. Добавить в программу возможность модифицировать передаваемое подписанное сообщение, тем самым нарушая его целостность. На этапе проверки подписи такого сообщения программа должна найти несоответствие хэш-сумм».

Вариант хэш-функции – 1: хэш-функция работы №3 с основной функцией $\sin(m)$ и выходом 32 бита.

В работе разработана программа ***signature.py***, которая подписывает текстовое сообщение, проверяет подпись и демонстрирует обнаружение модификации подписанного сообщения. Выполнение пунктов задания подтверждено экранами режимов программы, таблицей модификаций подписанного сообщения и автоматическим тестом на 1000 случайных сообщений.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

ЭЦП – число S , вычисленное из сообщения M закрытым ключом отправителя и проверяемое его открытым ключом. ЭЦП предназначена для обеспечения аутентичности (подтверждение, что сообщение подписано именно данным отправителем), целостности (невозможность изменения сообщения без обнаружения) и неотказуемости (отправитель не может отказаться от авторства).

Схема на основе RSA использует ключи из работы №2: модуль $n = p \cdot q$, где p и q – простые числа, функцию Эйлера $\phi = (p - 1)(q - 1)$, закрытый ключ d и открытый ключ e (вместе с модулем – пары $\{d, n\}$ и $\{e, n\}$), связанные условием $e \cdot d \equiv 1 \pmod{\phi}$. Отправитель подписывает хэш-сумму сообщения закрытым ключом и передаёт пару (M, S) :

$$h = H(M), \quad S = h^d \bmod n, \quad (1.1)$$

где

h – хэш-сумма сообщения;

H – хэш-функция;

M – сообщение;

S – подпись;

d – закрытый ключ;

n – модуль из алгоритма RSA.

Получатель вычисляет хэш-сумму сообщения и дешифрует подпись открытым ключом:

$$h' = H(M), \quad h'' = S^e \bmod n, \quad (1.2)$$

где

h' – хэш-сумма полученного сообщения;

h'' – хэш-сумма, восстановленная из подписи;

e – открытый ключ.

Затем получатель сравнивает h' и h'' : если они совпадают, подпись верна. Корректность схемы следует из $e \cdot d \equiv 1 \pmod{\phi}$ и теоремы Эйлера при $\text{НОД}(h, n) = 1$ (для h , кратных p или q , – из китайской теоремы об остатках): $S^e \equiv h^{d \cdot e} \equiv h \pmod{n}$, то есть $h'' = h \bmod n$. Отсюда условие применимости схемы $h < n$: иначе восстанавливается не h , а его остаток по модулю n .

Подписывается не сообщение, а его хэш-сумма фиксированной длины, которая укладывается в один блок RSA. Стойкость RSA-части определяется трудностью факторизации n , стойкость к подмене – односторонностью и стойкостью к коллизиям хэш-функции.

2 ОПИСАНИЕ АЛГОРИТМА И ПРОГРАММЫ

2.1 Постановка задачи

По заданию программа должна подписывать текст ASCII по формуле (1.1), проверять подпись по формуле (1.2) и модифицировать подписанное сообщение с последующей проверкой. Методические указания не определяют ряд деталей, поэтому приняты следующие допущения:

1. Хэш-функция – **my_hash** из работы №3, вариант $\sin(\mathbf{m})$, выход 32 бита; RSA – модуль **rsa.py** из работы №2 (простые **p** и **q**, выбираемый закрытый ключ **d**, открытый ключ $e = d^{-1} \bmod \varphi$).
2. Условие $h < n$ обеспечивается тем, что простые числа **p** и **q** берутся из диапазона $[2^{17}, 2^{18})$: тогда $n \geq 2^{34} > 2^{32} > h$; ключи примера из методических указаний к работе №2 ($n = 33$) для ЭЦП непригодны, что показывает отдельный режим.
3. Ключи проверяются на простоту **p** и **q** и условия $p \neq q$, $p \cdot q > 2^{32}$, $1 < d < \varphi$, $\text{НОД}(d, \varphi) = 1$, что обеспечивает существование **e** и корректность проверки.
4. Сообщение – строка любой длины из символов с кодами 0–127, включая пустую; символы вне ASCII отклоняются с повтором ввода.
5. Модификация – модель злоумышленника в канале связи: замена, вставка, удаление символа или перестановка соседних символов, вручную или случайно; подпись **S** при этом не пересчитывается.
6. Проверка выполняется для текущей пары (**M**, **S**), для изменённого сообщения с прежней подписью **S** и для пары, введённой вручную; при $h'' \geq 2^{32}$ значение **S** не является подписью под данным ключом, так как 32-битная хэш-сумма не может принять такое значение.
7. Правильность подтверждается автоматическим тестом: 1000 случайных сообщений длиной от 0 до 50 символов должны проходить проверку нетронутыми и отклоняться после модификации.

2.2 Описание алгоритма

Подпись формируется по формуле (1.1): хэш-сумму **h** вычисляет функция **my_hash** работы №3, возведение в степень по модулю – функция быстрого возведения в степень **modpow** работы №2. Проверка выполняется по формуле (1.2) со сравнением **h'** и **h''**.

Обнаружение модификации. Изменённое сообщение **M'** даёт $h' = H(M')$, тогда как $h'' = H(M)$ восстанавливается из неизменной подписи **S**. Подпись сходится только при коллизии $H(M') = H(M)$; для 32-битной хэш-суммы вероятность этого при случайном изменении равна 2^{-32} , а число различающихся бит **h'** и **h''** в среднем близко к 16 (лавинный эффект работы №3).

Псевдокод основных функций программы приведён на рисунке 2.1.

<pre> sign(M, d, n): h ← my_hash(M) если h ≥ n: ошибка S ← modpow(h, d, n) вернуть (h, S) </pre>	<pre> verify(M, S, e, n): h' ← my_hash(M) h'' ← modpow(S, e, n) вернуть (h', h'', h' = h'') </pre>
<pre> modify(M, вид, позиция, символ): replace: M[позиция] ← символ insert: вставить символ delete: удалить M[позиция] swap: M[i] ↔ M[i+1] S не пересчитывается </pre>	<pre> make_keys(p, q, d): p, q – простые из [2^17, 2^18) (или заданные) проверки: p ≠ q, p·q > 2^32, 1 < d < φ, НОД(d, φ) = 1 e ← d^(-1) mod φ (расширенный алгоритм Евклида) вернуть {e, n}, {d, n} </pre>

Рисунок 2.1 – Псевдокод основных функций

2.3 Описание программы

Программа **signature.py** написана на языке Python 3 с использованием только стандартной библиотеки и подключает модули предыдущих работ по относительному пути: **lab2/rsa.py** (возведение в степень по модулю, проверка простоты, генерация простых чисел, ключи, ввод чисел и текста ASCII) и **lab3/myhash.py** (хэш-функция, шестнадцатеричный вывод, расстояние Хэмминга). Запуск выполняется из корня репозитория командой «python3 lab4/signature.py»; аргумент «sin», «cos» или «xor» выбирает вариант хэш-функции. Листинг программы приведён в приложении 1.

Ядро программы составляют функции **sign**, **verify** (возвращает **h'**, **h''** и результат сравнения), **modify** (четыре вида модификации без пересчёта **S**), **make_keys** (формирование ключей с проверками) и **self_test** (автоматический тест). Состояние сеанса – ключи, сообщение, подпись, изменённое сообщение – хранит класс **State**; режимы меню 1–5 и 7 выполняют функции с префиксом **mode_**, режим 6 – функция **self_test**.

Главное меню программы содержит семь режимов и команду выхода «0». Режимы 1 и 2 задают ключи RSA (случайные или введённые вручную) и сообщение (с клавиатуры или случайное из печатных символов ASCII); режим 3 подписывает сообщение и выводит **h**, **S** и пару (**M**, **S**); режим 4 проверяет текущую пару, изменённое сообщение с той же подписью **S** или введённые **M** и **S**; режим 5 изменяет подписанное сообщение и проверяет его; режим 6 запускает автоматический тест; режим 7 демонстрирует условие $h < n$ на ключах $n = 33$ из примера методических указаний к работе №2. Режимы 3–5 без ключей, режимы 3 и 5 без сообщения и режим 5 без подписи выводят подсказку, а режим 4 при отсутствии подписи сразу запрашивает **M** и **S**; ошибки ввода (не простое **p**, $p \cdot q < 2^{32}$, символ вне ASCII, **S** вне $[0, n - 1]$) сообщаются с повтором ввода; сочетания клавиш «Ctrl+C» и «Ctrl+D» завершают программу.

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.1 Ключи RSA и сообщение ASCII

Для экранов режимов 1–5 ключи заданы вручную в режиме 1 (главное меню и режим 1 показаны на рисунке 3.1): $p = 131101$, $q = 262139$, $d = 1234567$; попытка задать $q = p$ отклонена. Пара $p = 61$, $q = 53$ из работы №2 также отклоняется, так как $n = 3233 < 2^{32}$. Случайные ключи дают, например, $p = 213029$, $q = 186859$, $n = 39806385911$.

```
Лабораторная работа №4. ЭЦП на RSA (ЛР2) и хэш-функции sin(m) (ЛР3)
1 – ключи RSA (случайные или вручную)
2 – сообщение (с клавиатуры или случайное)
3 – подписать сообщение
4 – проверить подпись
5 – изменить подписанное сообщение и проверить
6 – автотест на 1000 сообщений
7 – демонстрация условия  $h < n$  (ключи  $n = 33$  из примера ЛР2)
0 – выход
Выбор: 1
Ключи: 1 – случайные ( $p, q$  из  $[2^{17}, 2^{18})$ ), 2 – ввести  $p, q, d$  вручную: 2
p (простое): 131101
q (простое,  $q \neq p$ ,  $p \cdot q > 2^{32}$ ): 131101
q = 131101: нужно простое, отличное от p, с  $p \cdot q > 2^{32}$  (иначе хэш  $\geq n$ ).
q (простое,  $q \neq p$ ,  $p \cdot q > 2^{32}$ ): 262139
d ( $1 < d < \phi$ ,  $\text{НОД}(d, \phi) = 1$ ; Enter – случайное): 1234567
p = 131101, q = 262139, n =  $p \cdot q = 34366685039$  ( $> 2^{32} = 4294967296$ ),  $\phi = 34366291800$ 
Открытый ключ {e, n} = {8601127903, 34366685039}, закрытый {d, n} = {1234567, 34366685039}
```

Рисунок 3.1 – Главное меню и режим 1: ключи RSA вручную с проверкой $q \neq p$

Пункт 2 задания подтверждает режим 2, показанный на рисунке 3.2: текст с пробелами, знаками «\$», «,», «#» и цифрами принят, символ «П» отклонён, случайное сообщение составлено из печатных символов ASCII.

```
Выбор: 2
Сообщение: 1 – ввести с клавиатуры, 2 – случайное из печатных ASCII: 1
Текст (ASCII, Enter – пустое): Привет
Символ «П» (код 1055) вне кодировки ASCII, повторите ввод.
Текст (ASCII, Enter – пустое): Pay 100$ to Bob, order #42
M = 'Pay 100$ to Bob, order #42' (26 символов)
Выбор: 2
Сообщение: 1 – ввести с клавиатуры, 2 – случайное из печатных ASCII: 2
Длина (1..200): 20
M = '>SE)0DAzkCS&<J:ELL2X' (20 символов)
```

Рисунок 3.2 – Режим 2: сообщение с клавиатуры (отказ при символе вне ASCII) и случайное

3.2 Формирование и проверка подписи

Пункты 1 и 3 задания подтверждают режимы 3 и 4, показанные на рисунке 3.3: хэш-сумма $h = 97bcf201$ меньше n , подпись $S = 17008916931$, проверка нетронутой пары сходится.

```
Выбор: 3
h = H(M) = 97bcf201 = 2545742337 (h < n: да)
S =  $h^d \bmod n = 17008916931$ 
Передаётся пара (M, S): ('Pay 100$ to Bob, order #42', 17008916931)
Выбор: 4
Проверить: Enter – текущую пару (M, S), m – изменённое сообщение с той же S, i – ввести M и S:
h' = H(M) = 97bcf201
h'' =  $S^e \bmod n = 97bcf201$ 
Подпись верна: хэш-суммы совпадают, сообщение подлинно и не изменено.
```

Рисунок 3.3 – Режимы 3–4: формирование подписи и проверка нетронутого сообщения

Проверка пары, введённой вручную (режим 4, ответ «i»), показана на рисунке 3.4: произвольное $S = 12345$ даёт $h'' = 20006972960$, что больше 2^{32} и невозможно для 32-битной хэш-суммы. Подпись, искажённая в одном разряде ($S + 1$), при такой же проверке даёт $h'' = 6072391386$ с тем же вердиктом.

```

Выбор: 4
Проверить: Enter – текущую пару (M, S), m – изменённое сообщение с той же S, i – ввести M и S: i
M (ASCII): Pay 100$ to Bob, order #42
S: 12345
h' = H(M)           = 97bcf201
h'' = S^e mod n     = 20006972960 – не помещается в 32 бита: S не является подписью под этим ключом
ПОДПИСЬ НЕВЕРНА: хэш-суммы не совпадают – подпись чужая или искажена.

```

Рисунок 3.4 – Режим 4: проверка пары (M, S) с чужой подписью

3.3 Модификация сообщения и автоматический тест

Пункт 4 задания подтверждает режим 5, показанный на рисунке 3.5: замена «1» на «9» превращает сумму 100\$ в 900\$, подпись не пересчитывается, и проверка находит несоответствие хэш-сумм. То же несоответствие выводит режим 4 при ответе «m».

```

Выбор: 5
Модификация: 1 – заменить символ, 2 – вставить, 3 – удалить, 4 – переставить соседние, Enter –
случайная: 1
Позиция (0..25, Enter – случайная): 4
Символ ASCII (Enter – случайный): 9
Было: 'Pay 100$ to Bob, order #42'
Стало: 'Pay 900$ to Bob, order #42' – замена символа '1' на '9' в позиции 4; подпись S =
17008916931 не менялась
Проверка изменённого сообщения: h' = 4cc35b97, h'' = 97bcf201 → НЕ СОВПАДАЮТ (21 бит из 32) –
целостность нарушена
Выбор: 4
Проверить: Enter – текущую пару (M, S), m – изменённое сообщение с той же S, i – ввести M и S: m
h' = H(M)           = 4cc35b97
h'' = S^e mod n     = 97bcf201
ПОДПИСЬ НЕВЕРНА: хэш-суммы не совпадают (различаются 21 бит из 32) – сообщение изменено или
подписано другим ключом.

```

Рисунок 3.5 – Режим 5: замена символа и проверка изменённого сообщения

Все модификации сообщения при той же подписи, включая замену символа (см. рис. 3.5), приведены в таблице 3.1: каждая обнаружена, подпись признана неверной.

Таблица 3.1 – Модификации подписанного сообщения при $S = 17008916931$, $h'' = 97bcf201$

Модификация	Сообщение после модификации	Хэш-сумма $h' = H(M')$	Различие h' и h'' , бит
Замена «1» → «9», позиция 4	Pay 900\$ to Bob, order #42	4cc35b97	21
Вставка «0» в позицию 7	Pay 1000\$ to Bob, order #42	dd516147	16
Удаление «1» из позиции 4	Pay 00\$ to Bob, order #42	31017195	16
Перестановка «4» и «2», позиции 24–25	Pay 100\$ to Bob, order #24	dc2cfe6d	12
Случайная: « » → «5», позиция 3	Pay5100\$ to Bob, order #42	6b5f035c	21

Автоматический тест (режим 6) на случайных ключах: 1000 нетронутых сообщений

длиной 0–50 символов из печатных ASCII прошли проверку, все 1000 изменённых отклонены.

Режим 7 показывает, почему нужны большие p и q : на ключах примера из методических указаний к работе №2 ($n = 33$) хэш-сумма $h = 3264597241$ не меньше n , и проверка нетронутости сообщения не сходится – восстанавливается лишь $h \bmod 33 = 4$. Оба режима показаны на рисунке 3.6.

```

Выбор: 6
Ключи: p = 241333, q = 251939, n = 60801194687 (> 2^32: да), e = 32935383707, d = 36875377235
Сообщений: 1000 (длина 0..50, все печатные ASCII)
Нетронутые прошли проверку: 1000/1000
Изменённые отклонены: 1000/1000
Выбор: 7
Ключи примера ЛР2: p = 3, q = 11, n = 33, e = 7, d = 3; сообщение 'Hello'
h = 3264597241 (32 бита) ≥ n = 33; S = h^d mod n = 31; S^e mod n = 4 = h mod n = 4
Проверка нетронутости сообщения: НЕ сошлась – при h ≥ n восстанавливается лишь h mod n.
Поэтому p, q берутся из [2^17, 2^18): n ≥ 2^34 > 2^32 ≥ h.

```

Рисунок 3.6 – Режимы 6–7: автоматический тест на 1000 сообщений и условие $h < n$ на ключах $n = 33$

3.4 Анализ результатов

Обнаружение модификации. Все пять модификаций из таблицы 3.1 и 1000 случайных модификаций автоматического теста обнаружены. Хэш-суммы различаются на 12–21 бит, в среднем на 17,2 бита, что согласуется с лавинным эффектом хэш-функции работы №3 (около 16 бит из 32). Модификация останется незамеченной только при коллизии $H(M') = H(M)$: вероятность этого для случайного изменения равна $2^{-32} \approx 2,3 \cdot 10^{-10}$, направленный поиск второго прообраза требует около 2^{32} вычислений хэш-функции.

Чужая подпись. Для S , не являющейся подписью, значение $h'' = S^e \bmod n$ распределено по всему диапазону $[0, n - 1]$; при $n = 34366685039$ оно меньше 2^{32} лишь с вероятностью $2^{32}/n = 0,125$, и даже тогда должно совпасть с h' . Оба примера подраздела 3.2 ($S = 12345$ и $S + 1$) дали $h'' > 2^{32}$, что программа отмечает отдельно.

Условие $h < n$. Результаты режимов 6 и 7 (см. рис. 3.6) подтверждают вывод раздела 1: при $n = 33$ проверка не сходится даже на нетронутом сообщении, при $n \geq 2^{34}$ сходится всегда (1000 из 1000). Диапазон $[2^{17}, 2^{18})$ взят с запасом: $n \geq 2^{34}$, тогда как 32-битной хэш-сумме достаточно любых простых с $p \cdot q > 2^{32}$ – это условие и проверяется при ручном вводе.

Таким образом, пункт 1 задания подтверждают режимы 1, 3 и 4 (см. рис. 3.1, 3.3), пункт 2 – режим 2 (см. рис. 3.2) и таблица 3.1, пункт 3 – режимы 3, 4 и 6 (см. рис. 3.3, 3.4, 3.6), пункт 4 – режимы 5 и 6 (см. рис. 3.5, 3.6) и таблица 3.1.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Для чего используется ЭЦП? Какие свойства имеет подписанное сообщение? ЭЦП удостоверяет авторство и неизменность электронного документа: договоров, платёжных поручений, обновлений программ, сертификатов. Подписанное сообщение обладает аутентичностью (подпись $S = h^d \bmod n$ мог вычислить только владелец d), целостностью (изменение M меняет h' , и проверка не сходится, см. таблицу 3.1) и неотказуемостью (автор не может отрицать подпись, проверяемую его открытым ключом). Конфиденциальности подпись не даёт: сообщение M передаётся открыто.

Вопрос 2. Каким образом ЭЦП выявляет модификацию подписанного сообщения? Проверяющий вычисляет $h' = H(M)$ по полученному сообщению и $h'' = S^e \bmod n$ по подписи; h'' равно хэш-сумме исходного сообщения, h' – хэш-сумме полученного. Любое изменение M из-за лавинного эффекта меняет около половины бит хэш-суммы, и $h' \neq h''$ (совпадение возможно лишь при коллизии хэш-функции с вероятностью 2^{-32}): замена одной цифры изменила 21 бит из 32 (см. рис. 3.5). Подделать подпись под новое сообщение нельзя без d , а подобрать сообщение с прежней хэш-суммой – без поиска второго прообраза.

Вопрос 3. Почему в ЭЦП шифруется не само сообщение, а его хэш-сумма? Будет ли работать ЭЦП, если шифровать само сообщение? Хэш-сумма имеет фиксированную длину и заведомо меньше n : подпись – одно возведение в степень, её размер не зависит от длины документа. Подпись самого сообщения формально работает, если число M меньше n , но на практике непригодна. Длинный текст пришлось бы делить на блоки, как при шифровании в работе №2, и подписывать каждый, поэтому подпись не короче документа (при посимвольных блоках работы №2 примерно впятеро длиннее: число до $n \approx 2^{35}$ на 7-битный символ), время растёт пропорционально числу блоков, а блоки можно переставлять, удалять и дублировать без нарушения проверки (перестановка из таблицы 3.1 осталась бы незамеченной). Кроме того, RSA мультипликативна: из подписей M_1 и M_2 получается подпись $M_1 \cdot M_2 \bmod n$, а любое S даёт «подписанное» сообщение $S^e \bmod n$. Хэширование закрывает эти уязвимости.

Вопрос 4. Что такое сертификат ЭЦП? Из чего он состоит и для чего он требуется? Сертификат – электронный документ, связывающий открытый ключ с владельцем и заверенный подписью удостоверяющего центра (УЦ). По формату X.509 он содержит серийный номер, сведения о владельце, открытый ключ и алгоритм, наименование УЦ, срок действия, область применения и подпись УЦ. Без сертификата проверяющий не знает, чей ключ проверяет: злоумышленник подменяет открытый ключ $\{e, n\}$ на свой $\{e', n'\}$ (атака «человек посередине» из работы №1) и выдаёт свои подписи за чужие. Доверие строится по цепочке от корневого ключа УЦ; в России сертификаты выдают аккредитованные УЦ по закону «Об электронной подписи».

Вопрос 5. Что такое радужные таблицы и как им противостоять? Радужная таблица – заранее вычисленная структура для обращения хэш-функции на ограниченном множестве входов (паролей): цепочки чередующихся хэширования и функций редукции, различных на каждом шаге цепочки (отсюда название), от которых хранятся лишь начало и конец; память обменивается на время поиска. Противостоят ей соль – случайная строка, уникальная для каждого пароля (таблицу пришлось бы строить для каждой соли заново; в работе №3 соль задаётся аргументом *salt*), медленные функции формирования ключа (PBKDF2, bcrypt, Argon2), секретный «перец» на стороне сервера и длинные пароли.

ЗАКЛЮЧЕНИЕ

В работе реализована электронная цифровая подпись на алгоритме RSA работы №2 и хэш-функции варианта $\text{sin}(m)$ работы №3: подпись формируется по формуле (1.1), проверка выполняется по равенству $H(M)$ и $S^e \bmod n$ согласно формуле (1.2). Программа *signature.py* генерирует и принимает ключи с проверками, подписывает любой текст ASCII, проверяет пары (M, S) , модифицирует подписанное сообщение четырьмя способами и обнаруживает изменение.

Пункты 1–4 задания выполнены: нетронутое сообщение проходит проверку, все модификации из таблицы 3.1 отклонены с расхождением хэш-сумм на 12–21 бит, 1000 случайных модификаций автоматического теста отклонены полностью (1000 из 1000). Тем самым цель работы достигнута.

Установлено условие применимости схемы $h < n$: для 32-битной хэш-суммы нужен модуль $n = p \cdot q > 2^{32}$ (генератор берёт p и q из $[2^{17}, 2^{18})$, $n \geq 2^{34}$); ключи примера с $n = 33$ непригодны.

Схема обнаруживает случайные и преднамеренные изменения, но учебные параметры не дают криптографической стойкости: модуль $n \approx 2^{35}$ факторизуется перебором делителей до $\sqrt{n} \approx 185\,000$ за доли секунды (атака из работы №2), после чего злоумышленник вычисляет d и подписывает любые сообщения; 32-битная хэш-сумма допускает поиск коллизии примерно за 2^{16} сообщений (работа №3), то есть подготовку двух документов с одной подписью. Практические системы используют модуль 2048 бит и хэш-сумму 256 бит.

ПРИЛОЖЕНИЕ 1

ЛИСТИНГ ПРОГРАММЫ

Листинг модуля *signature.py* приведён на рисунке П1.1. Используемые им модули *lab2/rsa.py* и *lab3/myhash.py* приведены в отчётах по работам №2 и №3.

```
#!/usr/bin/env python3
"""Лабораторная работа №4. Реализация электронной цифровой подписи.

Подпись строится из программ предыдущих работ: RSA (lab2/rsa.py) и собственной
хэш-функции (lab3/myhash.py). Схема методички:
    формирование:  $h = H(M)$ ;  $S = h^d \bmod n$ ; передаётся пара (M, S)
    проверка:  $h' = H(M)$ ;  $h'' = S^e \bmod n$ ; подпись верна, если  $h' = h''$ 
Закрывающий ключ {d, n} есть только у автора (аутентичность, неотказуемость), открытый
{e, n} – у всех (проверяемость). Любое изменение M меняет h' – подпись не сходится
(целостность).

Условие корректности, которое нужно учесть при выборе ключей:  $h < n$ , иначе  $S^e \bmod n = h \bmod n \neq h$ 
и подпись не сходится даже на нетронutom сообщении. Хэш 32-битный, поэтому
простые p, q берутся из  $[2^{17}, 2^{18})$ :  $n = p \cdot q \geq 2^{34} > 2^{32}$ .

Режимы: ключи (случайно / вручную), ввод сообщения, подпись, проверка пары (M, S),
модификация сообщения с последующей проверкой, автотест, демонстрация условия  $h < n$ .

Только стандартная библиотека. Запуск: python3 signature.py [sin|cos|xor]
"""

import os
import random
import sys

BASE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(BASE, "..", "lab2")) # rsa.py – лабораторная №2
sys.path.insert(0, os.path.join(BASE, "..", "lab3")) # myhash.py – лабораторная №3
import rsa # noqa: E402 (modpow, is_prime, gcd, gen_prime, key_errors, make_keys, read_int,
    read_text)
import myhash # noqa: E402 (my_hash, to_hex, hamming, VARIANTS)

P_MIN, P_MAX = 2 ** 17, 2 ** 18 - 1 # простые для  $n > 2^{32}$  (хэш 32-битный)
N_MIN = 2 ** 32 #  $h < 2^{32} \leq n$  – условие корректности проверки
EXAMPLE_KEYS = (rsa.EXAMPLE["p"], rsa.EXAMPLE["q"], rsa.EXAMPLE["d"]) # пример ЛР2 ( $n = 33$ ) –
    демонстрация  $h \geq n$ 
ASCII_PRINTABLE = [chr(c) for c in range(32, 127)]

# --- Подпись и проверка -----
def sign(message, d, n, variant):
    """Формирование ЭЦП:  $h = H(M)$ ,  $S = h^d \bmod n$ . Возвращает (h, S)."""
    h = myhash.my_hash(message, variant=variant)
    if h >= n:
        raise ValueError(f"хэш  $h = \{h\} \geq n = \{n\}$ : подпись не проверится, нужны  $p, q \geq 2^{17}$ ")
    return h, rsa.modpow(h, d, n)

def verify(message, signature, e, n, variant):
    """Проверка ЭЦП:  $h' = H(M)$ ,  $h'' = S^e \bmod n$ . Возвращает (h', h'', совпали ли)."""
    h1 = myhash.my_hash(message, variant=variant)
    h2 = rsa.modpow(signature, e, n)
    return h1, h2, h1 == h2

def modify(message, kind, position=None, char=None, rng=random):
    """Нарушение целостности: replace / insert / delete / swap. Возвращает (M', описание).
    Подпись при этом НЕ пересчитывается – это и есть модель злоумышленника в канале."""
    if not message and kind != "insert":
        kind = "insert"
    if position is None:
        position = rng.randrange(len(message) + 1) if kind == "insert" else (rng.randrange(len(
            message)) if message else 0)
    if kind == "replace":
        old = message[position]
        new = char if char is not None and char != old else rng.choice([c for c in ASCII_PRINTABLE
            if c != old])
        return message[:position] + new + message[position + 1:], f"замена символа {old!r} на
            {new!r} в позиции {position}"
```

```

if kind == "insert":
    new = char if char is not None else rng.choice(ASCII_PRINTABLE)
    position = min(position, len(message))
    return message[:position] + new + message[position:], f"вставка символа {new!r} в позицию {position}"
if kind == "delete":
    return message[:position] + message[position + 1:], f"удаление символа {message[position]!r} из позиции {position}"
if kind == "swap":
    if len(message) < 2:
        return modify(message, "insert", rng=rng)
    position = min(position, len(message) - 2)
    if message[position] == message[position + 1]:
        return modify(message, "replace", position, rng=rng)
    swapped = message[:position] + message[position + 1] + message[position] + message[position + 2:]
    return swapped, f"перестановка символов {message[position]!r} и {message[position + 1]!r} (позиции {position}, {position + 1})"
raise ValueError(f"неизвестный вид модификации: {kind}")

def make_keys(p=None, q=None, d=None):
    """Ключи RSA средствами ЛР2 с диапазоном простых под 32-битный хэш."""
    if p is None:
        p = rsa.gen_prime(P_MIN, P_MAX)
        q = rsa.gen_prime(P_MIN, P_MAX)
        while q == p:
            q = rsa.gen_prime(P_MIN, P_MAX)
    errors = rsa.key_errors(p, q, d)
    if p * q < N_MIN:
        errors.append(f"n = p*q = {p * q} < 2^32: 32-битный хэш не поместится в блок")
    if errors:
        raise ValueError("; ".join(errors))
    (e, n), (d, n), phi = rsa.make_keys(p, q, d)
    return p, q, phi, (e, n), (d, n)

def self_test(variant, count=1000, seed=2026, verbose=True):
    """count случайных сообщений: нетронутые должны проходить проверку, изменённые — нет."""
    rng = random.Random(seed)
    saved = random.getstate()
    random.seed(seed)
    # gen_prime и make_keys из ЛР2 используют глобальный random
    p, q, phi, (e, n), (d, _) = make_keys()
    random.setstate(saved)
    ok_intact = ok_detected = 0
    for _ in range(count):
        message = "".join(rng.choice(ASCII_PRINTABLE) for _ in range(rng.randint(0, 50)))
        h, s = sign(message, d, n, variant)
        ok_intact += verify(message, s, e, n, variant)[2]
        changed, _ = modify(message, rng.choice(["replace", "insert", "delete", "swap"]), rng=rng)
        ok_detected += not verify(changed, s, e, n, variant)[2]
    if verbose:
        print(f" Ключи: p = {p}, q = {q}, n = {n} (> 2^32: {'да' if n > N_MIN else 'нет'}), e = {e}, d = {d}")
        print(f" Сообщений: {count} (длина 0..50, все печатные ASCII)")
        print(f" Нетронутые прошли проверку: {ok_intact}/{count}")
        print(f" Изменённые отклонены: {ok_detected}/{count}")
    return ok_intact == count and ok_detected == count

# --- Меню -----
class State:
    keys = None # (p, q, phi, (e, n), (d, n))
    message = None
    signature = None # S для message
    modified = None # изменённое сообщение (проверяется с тем же S)

def show_keys(st):
    p, q, phi, (e, n), (d, _) = st.keys
    print(f" p = {p}, q = {q}, n = p*q = {n} ({'>' if n > N_MIN else '<'} 2^32 = {N_MIN}), φ = {phi}")
    print(f" Открытый ключ {{e, n}} = {{{e}, {n}}}, закрытый {{d, n}} = {{{d}, {n}}}")

def mode_keys(st):
    choice = input(" Ключи: 1 — случайные (p, q из [2^17, 2^18)), 2 — ввести p, q, d вручную: ")
    choice = choice.strip()

```

```

if choice == "2":
    p = rsa.read_int(" p (простое): ", lambda v: None if rsa.is_prime(v) else f"p = {v} не простое")
    q = rsa.read_int(" q (простое, q ≠ p, p·q > 2^32): ",
                    lambda v: (None if rsa.is_prime(v) and v != p and p * v > N_MIN else
                                f"q = {v}: нужно простое, отличное от p, с p·q > 2^32 (иначе хэш ≥ n)"))
    phi = (p - 1) * (q - 1)
    d = rsa.read_int(" d (1 < d < φ, НОД(d, φ) = 1; Enter – случайное): ",
                    lambda v: None if 1 < v < phi and rsa.gcd(v, phi) == 1 else "d должно лежать в (1, φ) и быть взаимно простым с φ",
                        allow_empty=True)
    st.keys = make_keys(p, q, d)
else:
    st.keys = make_keys()
st.signature = st.modified = None
show_keys(st)

def mode_message(st):
    choice = input(" Сообщение: 1 – ввести с клавиатуры, 2 – случайное из печатных ASCII: ").strip()
    if choice == "2":
        length = rsa.read_int(" Длина (1..200): ", lambda v: None if 1 <= v <= 200 else "нужна длина от 1 до 200")
        st.message = "".join(random.choice(ASCII_PRINTABLE) for _ in range(length))
    else:
        st.message = rsa.read_text(" Текст (ASCII, Enter – пустое): ")
    st.signature = st.modified = None
    print(f" M = {st.message!r} ({len(st.message)} символов)")

def need(st, keys=True, message=False, signature=False):
    if keys and st.keys is None:
        print(" Сначала задайте ключи (пункт 1)."); return False
    if message and st.message is None:
        print(" Сначала введите сообщение (пункт 2)."); return False
    if signature and st.signature is None:
        print(" Сначала подпишите сообщение (пункт 3)."); return False
    return True

def mode_sign(st, variant):
    if not need(st, message=True):
        return
    _, _, _, (e, n), (d, _) = st.keys
    h, s = sign(st.message, d, n, variant)
    st.signature, st.modified = s, None
    print(f" h = H(M) = {myhash.to_hex(h)} = {h} (h < n: {'да' if h < n else 'нет'})")
    print(f" S = h^d mod n = {s}")
    print(f" Передаётся пара (M, S): ({st.message!r}, {s})")

def mode_verify(st, variant):
    if not need(st):
        return
    _, _, _, (e, n), _ = st.keys
    if st.signature is not None:
        raw = input(" Проверить: Enter – текущую пару (M, S), m – изменённое сообщение с той же S, i – ввести M и S: ").strip().lower()
    else:
        raw = "i"
    if raw == "m":
        if st.modified is None:
            print(" Изменённого сообщения нет – сначала пункт 5."); return
        message, s = st.modified, st.signature
    elif raw == "i":
        message = rsa.read_text(" M (ASCII): ")
        s = rsa.read_int(" S: ", lambda v: None if 0 <= v < n else f"S должно лежать в [0, n-1] = [0, {n - 1}]")
    else:
        message, s = st.message, st.signature
    h1, h2, ok = verify(message, s, e, n, variant)
    print(f" h' = H(M) = {myhash.to_hex(h1)}")
    if h2 >= 2 ** 32:
        print(f" h' = S^e mod n = {h2} – не помещается в 32 бита: S не является подписью под этим ключом")
    else:
        print(f" h' = S^e mod n = {myhash.to_hex(h2)}")

```

```

if ok:
    print(" Подпись верна: хэш-суммы совпадают, сообщение подлинно и не изменено.")
elif h2 < 2 ** 32:
    print(f" ПОДПИСЬ НЕВЕРНА: хэш-суммы не совпадают (различаются {myhash.hamming(h1, h2)} бит из 32) – "
          "сообщение изменено или подписано другим ключом.")
else:
    print(" ПОДПИСЬ НЕВЕРНА: хэш-суммы не совпадают – подпись чужая или искажена.")

def mode_modify(st, variant):
    if not need(st, message=True, signature=True):
        return
    kinds = {"1": "replace", "2": "insert", "3": "delete", "4": "swap"}
    choice = input(" Модификация: 1 – заменить символ, 2 – вставить, 3 – удалить, 4 – переставить соседние, Enter – случайная: ").strip()
    kind = kinds.get(choice) or random.choice(list(kinds.values()))
    position = char = None
    if choice in kinds and st.message:
        last = len(st.message) if kind == "insert" else len(st.message) - 1
        position = rsa.read_int(f" Позиция (0..{last}, Enter – случайная): ",
                                lambda v: None if 0 <= v <= last else "позиция вне сообщения",
                                allow_empty=True)
        if kind in ("replace", "insert"):
            typed = rsa.read_text(" Символ ASCII (Enter – случайный): ")
            char = typed[:1] or None
        st.modified, description = modify(st.message, kind, position, char)
        print(f" Было: {st.message!r}")
        print(f" Стало: {st.modified!r} – {description}; подпись S = {st.signature} не менялась")
        _, _, _, (e, n), _ = st.keys
        h1, h2, ok = verify(st.modified, st.signature, e, n, variant)
        print(f" Проверка изменённого сообщения: h' = {myhash.to_hex(h1)}, h'' = {myhash.to_hex(h2)} → "
              "f'{'подпись верна (?!)'} if ok else f'НЕ СОВПАДАЮТ ({myhash.hamming(h1, h2)} бит из 32) – целостность нарушена'}")

def mode_demo_small_n(variant):
    """Почему нужен n > 2^32: ключи примера методички ЛР2 (n = 33) не проверяют даже нетронутое сообщение."""
    p, q, d = EXAMPLE_KEYS
    (e, n), (d, _), _ = rsa.make_keys(p, q, d, check=False)
    message = "Hello"
    h = myhash.my_hash(message, variant=variant)
    s = rsa.modpow(h, d, n)
    h2 = rsa.modpow(s, e, n)
    print(f" Ключи примера ЛР2: p = {p}, q = {q}, n = {n}, e = {e}, d = {d}; сообщение {message!r}")
    print(f" h = {h} (32 бита) ≥ n = {n}; S = h^d mod n = {s}; S^e mod n = {h2} = h mod n = {h % n}")
    print(f" Проверка нетронутого сообщения: {'сошлась' if h2 == h else 'НЕ сошлась'} – при h ≥ n восстанавливается лишь h mod n.")
    print(f" Поэтому p, q берутся из [2^17, 2^18): n ≥ 2^34 > 2^32 ≥ h.")

def main():
    variant = sys.argv[1].lower() if len(sys.argv) > 1 else "sin"
    variant = {"1": "sin", "2": "cos", "3": "xor"}.get(variant, variant)
    if variant not in myhash.VARIANTS:
        print(f"Неизвестный вариант хэш-функции «{sys.argv[1]}»: допустимы sin, cos, xor.")
        return
    st = State()
    menu = (f"\nЛабораторная работа №4. ЭЦП на RSA (ЛР2) и хэш-функции {variant}(m) (ЛР3)\n"
           " 1 – ключи RSA (случайные или вручную)\n"
           " 2 – сообщение (с клавиатуры или случайное)\n"
           " 3 – подписать сообщение\n"
           " 4 – проверить подпись\n"
           " 5 – изменить подписанное сообщение и проверить\n"
           " 6 – автотест на 1000 сообщений\n"
           " 7 – демонстрация условия h < n (ключи n = 33 из примера ЛР2)\n"
           " 0 – выход")
    actions = {"1": lambda: mode_keys(st), "2": lambda: mode_message(st), "3": lambda: mode_sign(st, variant),
              "4": lambda: mode_verify(st, variant), "5": lambda: mode_modify(st, variant),
              "6": lambda: self_test(variant), "7": lambda: mode_demo_small_n(variant)}
    try:
        while True:
            print(menu)
            choice = input("Выбор: ").strip()

```

```
        if choice == "0":
            break
        if choice in actions:
            try:
                actions[choice]()
            except ValueError as err:
                print(f" Ошибка: {err}.")
        else:
            print(" Нет такого пункта меню.")
    except (EOFError, KeyboardInterrupt):
        print("\nВыход.")

if __name__ == "__main__":
    main()
```

Рисунок П1.1 – Листинг модуля signature.py